

VITyarthi

EVALUATED COURSE PROJECT

ROCK PAPER SCISSORS

A Console-Based Python Game

A small, interactive Python project built around user input, random choice, validation, conditional logic and replay control.

Student Name	RANI PARMAR
Registration Number	26MIM10109
Course / Subject	Introduction to problem solving and programming
Faculty Name	Devendra Kumar Vedi

Prepared as a course project report based on the working program developed in Python.

TABLE OF CONTENTS

No.	Section	Page
1	Introduction and Project Overview	3
2	Problem Statement and Objectives	4
3	Scope, Users and Functional Requirements	5
4	Non-Functional Requirements and Constraints	6
5	System Design and Architecture	7
6	Workflow, Use Cases and Sequence	8
7	Algorithm and Program Logic	9
8	Implementation Details	10
9	Testing and Results	11
10	Challenges, Learnings and Future Enhancements	12

Executive Summary

Rock Paper Scissors is a console-based Python game in which the player chooses one of three options - rock, paper or scissor - while the computer makes a random choice. The program validates the player input, displays both choices, compares them through conditional statements, and reports whether the round is a win, loss or tie. The player can then decide whether to start another round or exit the program.

The project is intentionally small, but it demonstrates several core programming ideas in one complete flow: variables, tuples, loops, user input, string methods, the random module, conditional branching and basic control of repeated execution. The report explains the project as it is actually implemented rather than introducing features that are not present in the program.

Project Snapshot

Item	Details
Language	Python 3
Interface	Command-line / terminal
External library	random (Python standard library)
Main data structure	Tuple of choices
Main control flow	while loops + if / elif / else
Persistence	Not required for this game

1. INTRODUCTION AND PROJECT OVERVIEW

1.1 Background

Rock Paper Scissors is a simple game that can be implemented with a small number of rules, which makes it useful for practicing the building blocks of a Python program. Instead of treating the exercise as a collection of separate syntax examples, this project connects the concepts into one running application that accepts input, makes a decision, shows an output and repeats until the player stops.

1.2 Purpose of the Project

The purpose of the project is to demonstrate how a basic Python program can turn a user action into a complete interactive round. The program uses the random module to create an unpredictable computer choice and uses conditional logic to determine the result. Input validation is included so that values outside the three supported choices are not processed as a game result.

1.3 Key Concepts Demonstrated

- Tuple: the available choices are stored together as ("rock", "paper", "scissor").
- Random selection: `random.choice(options)` selects one item from the tuple for the computer.
- Input handling: `input(...).lower()` accepts user text and normalizes it to lowercase.
- Loops: an outer loop controls repeated rounds and an inner loop keeps asking until a valid choice is entered.
- Conditional logic: `if / elif / else` blocks implement tie, win and loss conditions.
- Boolean control: the running flag controls whether another round should be started.

1.4 Project Outcome

The final outcome is a lightweight terminal game that starts from a single Python file, needs no third-party packages, and can be understood and demonstrated directly from the command line. The project therefore focuses on clarity, correct control flow and a complete user interaction cycle rather than on external services or a graphical interface.

2. PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Statement

A beginner-friendly programming exercise often teaches input, conditions and loops separately. The problem addressed by this project is to combine those concepts into a single interactive program that responds to a player, generates a computer choice, applies a fixed set of rules and allows the user to repeat the process without restarting the program manually.

The system must also handle an invalid choice without treating it as a valid move. The program therefore needs a small validation loop before the result calculation begins.

2.2 Objectives

- To build a working Rock Paper Scissors game using Python.
- To generate a computer move randomly from the allowed choices.
- To accept and normalize user input from the terminal.
- To reject unsupported choices and request valid input again.
- To compare the two choices using clear conditional logic.
- To display an understandable round result: tie, win or loss.
- To allow the player to decide whether to continue playing.
- To keep the implementation simple enough to be read, tested and explained by the student.

2.3 Success Criteria

Criteria	Expected Result
Valid input	rock, paper or scissor is accepted.
Invalid input	Program shows "Invalid Input" and asks again.
Computer choice	One of the three supported options is selected.
Round result	Tie, win or lose message is displayed.
Replay	The user can continue or stop the game.
Exit	Program ends with a closing message.

3. SCOPE, USERS AND FUNCTIONAL REQUIREMENTS

3.1 Scope

The scope is limited to a single-player terminal game against a computer-generated move. The system does not store user accounts, scores or history, and it does not use a graphical interface. Keeping the scope narrow makes the program suitable for demonstrating core Python logic without adding unrelated technologies.

3.2 Target User

The immediate user is a student or general computer user who wants to play a quick round through a terminal. From a course-project perspective, the primary audience is the evaluator who needs to see a complete, executable example of Python control flow.

3.3 Functional Requirements

ID	Function	Requirement
FR-01	Start the game	The program starts from the command line and presents a choice prompt.
FR-02	Generate computer choice	The computer selects rock, paper or scissor using <code>random.choice</code> .
FR-03	Accept player choice	The user can enter a text choice, and it is converted to lowercase.
FR-04	Validate choice	Unsupported input is reported as invalid and the player is asked again.
FR-05	Determine result	The program identifies a tie, a player win, or a player loss.
FR-06	Show round details	The selected player and computer moves are printed to the console.
FR-07	Replay or exit	The player can enter <code>y</code> to continue or another response to stop.
FR-08	Close cleanly	The program prints a closing message before ending.

3.4 Inputs and Outputs

Input: player choice and replay choice. Internal input: random computer choice. Output: player move, computer move, result message, invalid-input feedback and final closing message.

4. NON-FUNCTIONAL REQUIREMENTS AND CONSTRAINTS

4.1 Non-Functional Requirements

Requirement	How it is addressed
Usability	Prompts should be clear enough for a first-time terminal user.
Correctness	The implemented rules should consistently map the two choices to tie, win or loss.
Reliability	Invalid user input should not crash the game; it should return to the input step.
Maintainability	The program should use straightforward names and a simple control-flow structure so changes are easy to make.
Performance	Each round should execute immediately with negligible processing time.
Portability	The program should run on a standard Python 3 environment without third-party packages.

4.2 Constraints

- The project is terminal-only; no GUI framework is used.
- Only three fixed choices are supported.
- There is no persistent score or player profile.
- The project depends on standard Python input/output and the random module.
- The replay response is simple: the program continues when the entered value equals y; otherwise it stops.

4.3 Error Handling Strategy

The main input-related error handled by the program is an invalid move. The condition player not in options prints an "Invalid Input" message and uses continue so that the inner validation loop asks for another value. This prevents unsupported text from reaching the result calculation.

4.4 Design Rationale

A minimal command-line architecture was selected because the educational value is in the logic itself. Adding a database, framework or interface layer would make the project larger without improving the core objective of demonstrating input, randomness, loops and conditions.

5. SYSTEM DESIGN AND ARCHITECTURE

The system follows a simple procedural architecture. The player input and computer choice meet at the validation and comparison stages. After the result is printed, the replay decision either starts another round or ends the program.

System Architecture - Rock Paper Scissors

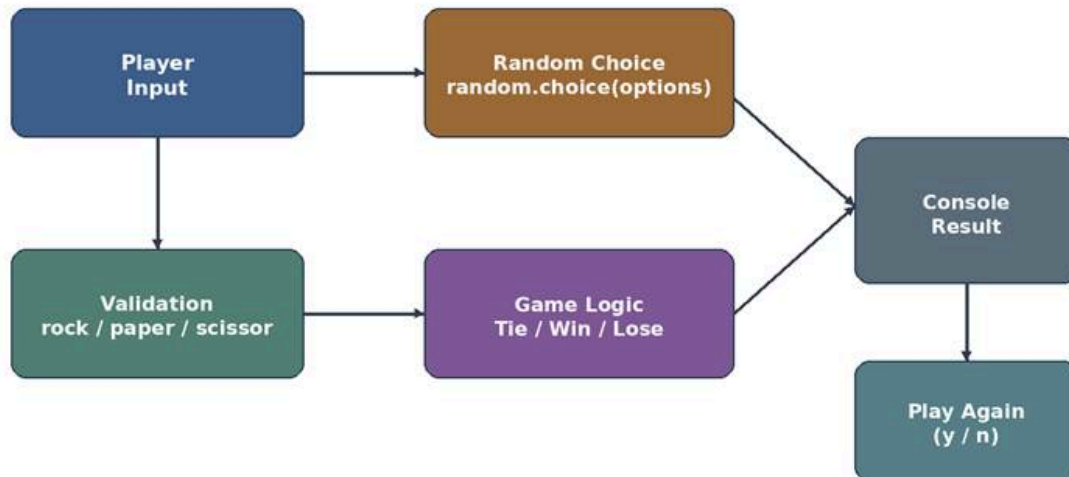


Figure 1. High-level architecture of the game

5.1 Main Components

Component	Responsibility
Input Layer	Reads and normalizes the player move and replay response.
Choice Set	Stores the three supported strings in a tuple.
Random Choice	Selects the computer move from the tuple.
Validation Loop	Rejects unsupported player input and repeats the prompt.
Result Logic	Checks tie and winning combinations; the remaining valid case is a loss.
Output Layer	Prints moves, result and closing message.
Replay Control	Updates the running flag according to the replay response.

5.2 Data Flow

The player provides a string. The string is converted to lowercase and compared with the tuple of valid options. The computer choice is generated independently. When validation passes, the two values are compared and a result string is printed. No external data source is required.

6. WORKFLOW, USE CASES AND SEQUENCE

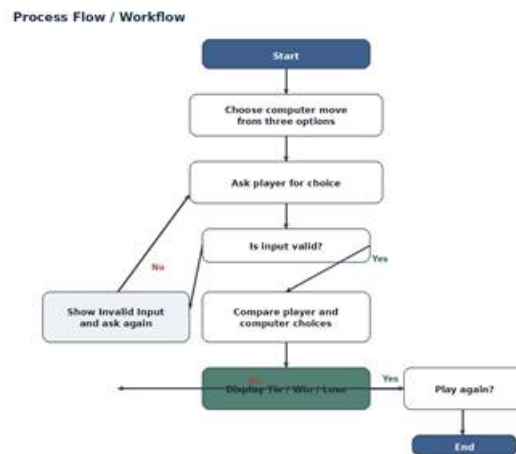


Figure 2. Process workflow

6.1 Primary Use Cases

- Enter a valid move or retry after invalid input
- View the round result
- Choose whether to play another round or exit

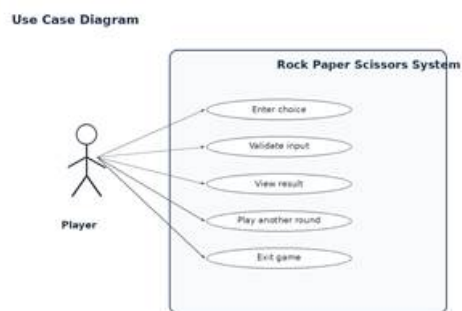


Figure 3. Use case view



Figure 4. One-round sequence

7. ALGORITHM AND PROGRAM LOGIC

7.1 Algorithm

1. Store rock, paper and scissor in the options tuple.
2. Set the running control variable to True.
3. While the game is running, choose one option randomly for the computer.
4. Set player to an empty value and start the validation loop.
5. Read the player input and convert it to lowercase.
6. If the input is not in options, print Invalid Input and continue the validation loop.
7. Once valid, print both choices and compare them.
8. If both values are equal, display a tie.
9. Otherwise check the three winning combinations; if none match, display a loss.
10. Ask whether to play again. Continue only when the response is y.
11. When the loop ends, print Thanks for Playing.

7.2 Core Decision Logic

```
if player == computer:
    print("Its Tie !")
elif player == "rock" and computer == "scissor":
    print("You won !")
elif player == "paper" and computer == "rock":
    print("You won !")
elif player == "scissor" and computer == "paper":
    print("You won !")
else:
    print("you lose !!")
```

7.3 Functional Structure

Component / Functional Structure

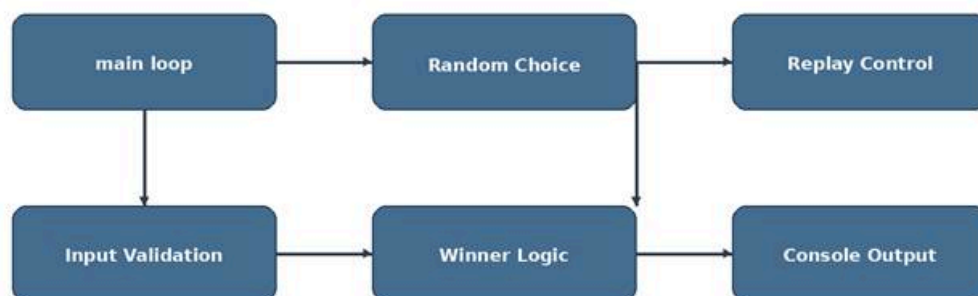


Figure 5. Functional/component view

8. IMPLEMENTATION DETAILS

8.1 Technologies Used

Technology	Use in the project
Python 3	Main programming language and runtime.
random module	Generates the computer's move.
Console I/O	Reads user choices and prints results.
Tuple	Stores the fixed set of options.

8.2 Important Code Elements

The options tuple acts as the single source of valid moves. The random.choice function prevents the computer move from being fixed. The lower() string method lets the user type mixed-case values while the rest of the logic works with lowercase text.

```
import random
options = ("rock", "paper", "scissor")
computer = random.choice(options)
player = input("Enter a choice(Rock,Paper,scissor): ").lower()
```

8.3 Validation Implementation

```
while player not in options:
    player = input("Enter a choice(Rock,Paper,scissor): ").lower()
    if player not in options:
        print("Invalid Input")
        continue
```

8.4 Replay Implementation

The outer loop is controlled by the running variable. After a round is complete, the program asks for a replay response. If the response is not y, running is set to False, which ends the outer loop and leads to the final closing message.

8.5 Execution

```
python rock_paper_scissors.py
```

No third-party installation is required. The project can be executed in a terminal with a standard Python 3 interpreter.

9. TESTING APPROACH AND RESULTS

9.1 Testing Method

Testing was treated as a set of manual functional checks. The aim was to exercise every meaningful branch in the game: valid input, invalid input, all winning combinations, tie, loss and both replay outcomes. The same input flow a user would follow in the terminal was used for the checks.

ID	Test Input / Situation	Expected Result	Status
TC-01	Enter rock when computer selects rock	Tie message shown	Pass
TC-02	Player rock, computer scissor	Player win message	Pass
TC-03	Player paper, computer rock	Player win message	Pass
TC-04	Player scissor, computer paper	Player win message	Pass
TC-05	Player rock, computer paper	Loss message	Pass
TC-06	Enter unsupported text	Invalid Input, then reprompt	Pass
TC-07	Replay input y	New round starts	Pass
TC-08	Replay input other than y	Program exits and prints closing message	Pass

9.2 Sample Result

```
ROCK PAPER SCISSORS
Enter a choice(Rock,Paper,scissor): rock
Player : rock
Computer : scissor
You won !
Play again ? (y/n) y
Enter a choice(Rock,Paper,scissor): paper
Player : paper
Computer : paper
Its Tie !
Play again ? (y/n) n
Thanks for Playing
```

Figure 6. Example terminal interaction

9.3 Testing Conclusion

The checks above cover the normal path and the important decision branches of the current implementation. In particular, the invalid-input case confirms that an unsupported move is not treated as a valid game choice.

10. CHALLENGES, LEARNINGS AND FUTURE ENHANCEMENTS

10.1 Challenges Faced

- Keeping the validation loop separate from the result logic so invalid input does not produce a fake result.
- Writing the win conditions correctly for all three possible winning combinations.
- Controlling the outer loop so the game can repeat but can also stop cleanly.
- Handling user input in a way that is simple enough for a terminal demonstration.

10.2 Learnings and Key Takeaways

- How tuples can hold a fixed collection of related choices.
- How `random.choice` can introduce non-deterministic computer behavior.
- How while loops can be used for both repetition and validation.
- How conditional statements translate a set of game rules into executable logic.
- Why checking user input before applying business logic makes a program safer and easier to understand.

10.3 Future Enhancements

- Add a running score for player wins, computer wins and ties.
- Add input normalization for alternate spellings or a menu-based numeric choice.
- Add a graphical interface using a suitable Python GUI library.
- Add persistent match history using a file or small database.
- Add a best-of-N game mode and a summary screen at the end.

10.4 Conclusion

The Rock Paper Scissors project demonstrates a complete and executable Python interaction from input to output. Its value comes from combining several basic concepts into one working application, while keeping the code small enough to inspect and explain. The current implementation meets the defined functional scope and provides a clear base for future extensions.